Тема 4

• член-функции – функции в тялото на структура/клас, които работят директно с член-данните на обекта

→ преобразуват се във външни функции с добавен първи параметър – константен указател към обекта, от който се извиква ↳ this

```
struct A {
  void f();        → void f(A* const this);
  void g() const;  → void g(const A* const this);
}
```

→ константни функции:
- → не променят член данните
- → могат да се извикват от константни обекти
- → могат да извикват само други константни ч.-ф.

• конструктор
- → член-функция, която се извиква при създаването на обекта
- → има същото име като класа/структурата
- → няма тип на връщане
- → конструктор без параметри се нарича default-ен
- → клас/структура може да има много конструктори (overloading)
- → ако няма други разписани конструктори, компилаторът създава default-ен автоматично (но не го създава, ако има други разписани конструктори)

```
struct A {
    A() { } → НЕ
    A() = default; → така компилаторът може да прави оптимизации
}

A arr[10]; → извиква A() 10 пъти
```
(и после ~A() 10 пъти в края на scope-а)

• деструктор

→ точно 1 във всеки клас/структура

→ отговаря за затварянето на външни ресурси

→ общото име като конструктора, но с ~ в началото

```
struct A {
    B b;
    C c;
}
```

→ ред на ~~викане~~ изпълнение при влагане:

- конструктори → B() C() A()
- деструктори → ~A() ~C() ~B()

→ конструкторът на A() е отворен за създаването на B и C

→ редът на извикване на конструкторите и деструкторите е важен, защото е възможно данните на различните член-данни да зависят едни от други

$\boxed{\text{new}}$ → заделя памет
↓
вика конструктор

$\boxed{\text{delete}}$ → освобождава памет
↓
вика деструктор

→ компилаторът генерира деструктор, ако не е разписан

конвертиращ конструктор (≤C++11)

конструктор, приемащ точно 1 параметър

```
struct A {
    int x;
    A(int) {}
};
```

↓ конв. конст.

→

```
int main() {
    int a = 10;
    A obj = a;
}
```

→ валиден синтаксис

```
struct B {
    int y;
    explicit B(int) {}
};
```

↓ констр. НЕ е конвертиращ

→

```
int main() {
    int b = 1;
    B obj = b;
    B obj2(b);
};
```

→ НЕ е валидно

→ валидно

- абстракция
  - → използване на нещо без да се интересуваме как работи
  - → скриване на ненужните детайли
- капсулация
  - → ограничаване на достъп
- модификатори за достъп
  - → private – имаме достъп до данните само в класа
  - → protected – имаме достъп до данните в класа и в наследниците му
  - → public – имаме достъп до данните навсякъде

• get и set член функции

→ get – връща копие/конст. реф./конст. указ. към избрана член-данна
– константни функции

→ set – позволява контролирана модификация на член-данни

• mutable член-данни

→ могат да бъдат променени дори и в f() const

→ не влияят на видимото състояние на обекта

→ добре е да се ползват само в краен случай

• Пример за class Person с име [20+1] и години ∈ [5; 90]:

```
const int nameMaxLen=20;
const int nameMinLen=2;
const int minAge=5;
const int maxAge=90;

bool isSmallLetter(char ch){
    return ch>='a' && ch<='z';
}

bool isCapitalLetter(char ch){
    return ch>='A' && ch<='Z';
}

bool containsOnlySmallLetters(const char* str){
    if(!str){return false;}
    ~~for(int~~
    size_t len=strlen(str);
```